Phase A — Foundations and data in

Phase A is Stage A of the *Working Paper Platform – Development Build Order v1.1* (steps 1–4), with story details in *User Stories v1.1*. Its goal: **a practice can log in, set up a client, pull its trial balance and see every account mapped.** It ends at milestone **M1 – Data in**.

This document covers what is built, how each part works, how to run and test it, and what is still open.

Step	Stories	Status
1 · Platform foundations	ADM-01, ADM-03, AUD-01	Done (MFA built, switched off until launch)
2 · Clients, engagements and periods	ENG-01, ENG-02, ADM-02 (first version), ENG-05 (basic)	Done, plus practice set-up wizard and client archive
3 · Trial balance import	DAT-02, DAT-01, DAT-06, DAT-07	CSV import and change detection done; **Xero / QuickBooks not built**
4 · Mapping, confidence gate, rules library	DAT-03, DAT-04, DAT-05, AUD-03, DAT-08, DAT-09	Done

---

Quick start

```
npm install
npm run dev
```

Open http://localhost:3000 (the dev server moves to 3001 if 3000 is taken).

- **New practice:** *Create a firm account* → six-step set-up wizard → app.
- **Seeded demo users:** `admin@blynt.co.uk`, `marcus.vance@blynt.co.uk` (preparer), `sarah.jenkins@blynt.co.uk` (reviewer), `david.thorne@blynt.co.uk` (partner). Password is `TEST_USER_PASSWORD` (default `admin123`) for users created before any password was stored.

```
npm test          # vitest (use --pool=threads on Windows if workers time out)
npx tsc --noEmit  # typecheck
npm run lint      # oxlint
```

Configuration

Variable	Default	Purpose
`AUTH_SECRET`	dev fallback	Signs session and MFA-challenge cookies. **Set a 32+ character value in production.**
`MFA_REQUIRED`	off	`true` makes TOTP mandatory for every sign-in (ADM-01). Off until launch.
`DATABASE_URL`	unset	Postgres for workspace state; without it state is kept in `.workspace_state.json` (or `/tmp` on serverless).
`LOCAL_STATE_FILE`	`.workspace_state.json`	Override the local state file path.
`TEST_USER_PASSWORD`	`admin123`	Password given to seeded users with no stored password.

How the system fits together

```
Browser (Next.js pages)                          Server (route handlers)
─────────────────────────                        ──────────────────────────
WorkspaceSync ── GET/PUT /api/v1/state ────────  state-guard.ts
  local db copy                                    firmSlice()     what this user may see
                                                   mergeFirmState() what this user may change
pages ──────────── /api/v1/* (fetch) ──────────  api-helpers.ts   getActorFromRequest / requireEngagementAccess
                                                 services.ts      clients, engagements, mapping, diffs
                                                 gate.ts          confidence gate + rules + AI log
                                                 invites.ts, onboarding.ts, totp.ts, auth.ts
                                                 store.ts ──  Postgres or local JSON file
```

- **One workspace document.** All data lives in one state object (`src/lib/server/db.ts`), persisted by `store.ts` with optimistic versioning.
- **Two ways to write.** Pages either edit their local copy and sync it through `/api/v1/state`, or call a dedicated API route. Anything that needs a server-side decision (sign-in, invites, imports, mapping corrections, archive, set-up) goes through an API route.
- **The guard is the security boundary.** Every sync is checked by `state-guard.ts`; it never trusts the browser. See Security rules.

Step 1 · Platform foundations

Data model

`practice (firm) > client > engagement > (period) > schedule (workpaper) > line`. Types are in `src/lib/types/models.ts`. Money is integer pence (`bigint`, stored as strings).

Sign-in and MFA

File	Role
`src/lib/server/auth.ts`	scrypt password hashes, signed session cookie (12 h), MFA challenge cookie (10 min)
`src/lib/server/totp.ts`	RFC 6238 TOTP (30 s, 6 digits, ±1 step drift)
`src/app/api/v1/auth/login`	Password check; issues a session, or a challenge when MFA applies
`src/app/api/v1/auth/mfa/setup`	Gives an enrolling user a new authenticator secret
`src/app/api/v1/auth/mfa/verify`	Checks the code, then issues the session

Flow with `MFA_REQUIRED=true`:

```
POST /auth/login ──  { mfa: "setup" | "verify" } + blynt_mfa cookie (not a session)
GET  /auth/mfa/setup ──  { secret, otpauth }            (first time only)
POST /auth/mfa/verify { code } ──  blynt_session cookie
```

A challenge token can never be used as a session (`verifySessionToken` rejects it). Failed passwords and codes lock out after repeated attempts. Login matches the exact email only — there are no fallback accounts or shared passwords.

Invites and roles

Roles: `preparer`, `reviewer`, `manager`, `partner`, `admin`.

- **Invite** (`POST /api/v1/invites`): admins, partners or the firm's creator. Returns a one-time link valid 7 days; only its SHA-256 is stored. There is no mail service yet, so the inviter sends the link.

- **Accept** (`/invite/<token>` → `POST /api/v1/invites/accept`): the person sets a password (same rules as sign-up) and the link is used up.
- **Assigned clients only:** members below partner may be limited to a client list (`Membership.client_ids`). Partners and admins see every client. Change with `PUT /api/v1/users/<id>/clients`.

Access is enforced in three places: `firmSlice` (what the browser receives), `requireEngagementAccess` (every `/engagements/<id>/*` route), and the client services (`getClients`, `getClient`, `createEngagement`).

Audit trail (AUD-01)

`db.logAudit(firmId, actor, action, entity, entityId, before, after, reason?)` records user, role, time, old value, new value and reason. Entries are frozen when written, and the sync guard refuses any edit or deletion and any entry not written as the signed-in user.

Step 2 · Clients, engagements and periods

- **Clients and engagements:** `POST /api/v1/clients`, `POST /api/v1/engagements`. Engagements can only be created for clients in your firm that you're assigned to and that aren't archived.
- **Period selector:** in the top bar (`src/components/layout/PeriodSelector.tsx`).

Practice set-up wizard (ADM-02 first version)

Shown once after sign-up at `/onboarding` (`src/app/onboarding/page.tsx`, server in `src/lib/server/onboarding.ts`). Adaptive: follow-up questions appear only when they apply, and a *What this changes in Blynt* panel explains each answer.

#	Step	Required	Saved to
1	Your practice: names, structure, Companies House no. (LLP/Ltd only), supervisory body, office, VAT	Yes	`firm.profile`
2	Your work: standards, services, bookkeeping software, size	Yes	`firm.settings.work`
3	How you review: sign-off levels, default standard, year end, tolerance, review threshold	Yes	`firm.settings.defaults`
4	Your team: invites	No	invites
5	First clients: sample company, CSV list, or by hand	No	clients
6	Review and finish	—	`firm.onboarding.completed_at`

API: `GET /api/v1/firm/onboarding`, `PUT /api/v1/firm/onboarding { step, data }`, `POST /api/v1/firm/onboarding/complete`. Only admins, partners and the firm's creator can run it; until it's finished they are sent to the wizard, while other members carry on working. Firms created before the wizard existed count as set up.

Client archive

Archive a client you no longer act for from the client list (`POST /api/v1/clients/<id>/archive { archived, reason? }`). Managers, partners, admins or the firm's creator.

- Nothing is deleted; it moves to the **Archived** tab and out of the client switcher.
- Its engagements become read-only, and new engagements and trial balance imports are refused until it's restored.
- Archive and restore are audited with the reason, and can't be done through a plain sync.

Sign-up rules

Shared by the form and the server (`src/lib/validation.ts`):

- **Email:** valid format (one `@`, dotted domain, no spaces or double dots); common domain typos get a suggestion.
- **Password:** 12+ characters, upper and lower case, a number, a symbol, and not the person's name or email. The form shows a live checklist and a confirm field.

Step 3 · Trial balance import

`POST /api/v1/engagements/<id>/imports { lines: [{ code, name, type, debit, credit }] }`

1. Rejects an unbalanced TB, returning the difference.
2. Stores an immutable snapshot with a SHA-256 of the raw lines.
3. Compares with the previous snapshot (DAT-06) and returns `changes`: account, prior and new net balance, difference, direction. This is how a late posting shows up on refresh.
4. Maps every line through the gate (step 4) and returns a summary: how many by rule, Ready, Check and Exception.

Transaction drill-down data (DAT-07) is modelled as `LedgerTransaction` with an `external_url` back to the ledger.

Step 4 · Mapping, confidence gate and rules library

One interface for every AI output (DAT-08, DAT-09, AUD-03)

`src/lib/server/gate.ts`:

```
runGated({ firmId, outputType, ruleType, keys, clientId, supplier, decode, ai })
// rules first (supplier > client > practice), then the AI proposal, then Ready / Check / Exception

learnFromCorrection(actor, { type, scope, pattern, target_value, client_id?, supplier_name? })
recordDecision(actor, { engagementId, outputType, itemRef, confidence, outcome, inputs })
accuracy(firmId, outputType, engagementId?)   // share accepted without change
approveRule(actor, ruleId)                     // managers approve practice rules
```

- **Thresholds** per output type (defaults: mapping Ready ≥ 90, Check ≥ 70), overridable in `firm.settings.gate`.
- **Auto-accept** is built but off for every output type.
- **Rule patterns:** typed keys (`code:4000`, `name:office rent`) match exactly; bare patterns with `*` match the name (`*rent*`).
- **Practice rules** from someone below manager stay inactive until a manager approves them (also enforced by the sync guard).
- A second output type (e.g. `extraction`) uses the same calls unchanged; this is covered by a test.

Mapping order (DAT-03 / DAT-04)

`services.suggestMapping` / `buildMapping`:

1. Client rule → 2. Practice rule → 3. Last year's confirmed mapping → 4. Name and code patterns (`src/lib/mapping/auto-mapper.ts`).

A rule match is confirmed with no review needed. Everything else is suggested and waits for a person.

Code versus name. A client's nominal code only matches the Blynt chart by chance, so the name decides:

Case	Example	Result
Name agrees with code	"Share Capital" on 3000	Code's account, 100%, Ready
Name disagrees	"Business Current Account" on 1200 (Trade Debtors)	Name's account, ≤ 80%, Check, with the clash in the reason
Name gives no clue	"Account XJ-9" on 1200	Code's account, 85%, Check

Corrections become rules (DAT-05)

`PATCH /api/v1/mappings/<id> { blynt_code }` confirms the mapping, logs `edited` or `accepted` in the AI action log, and creates a client rule (`code:<source_code>`). The same account maps automatically on that client's next import, and nowhere else.

Accuracy (AUD-03)

`GET /api/v1/ai/accuracy?engagementId=&type=mapping` → `{ decided, accepted_unchanged, edited, rejected, rate }`. This is where the 90% target is measured, and later what decides when auto-accept can be switched on.

Security rules

Enforced by `checkRules` in `src/lib/server/state-guard.ts` on every sync, and by the API routes:

Rule	Where
A user only sees and changes their own firm; below partner, only assigned clients	`firmSlice`, `requireEngagementAccess`, client services
Server-managed data (firms, users, memberships, credentials, invites, templates) can't be changed by a sync	`SERVER_MANAGED`
Audit log is append-only and written as yourself	`checkRules`
Locked engagements and archived clients' engagements are read-only	`checkRules`
Only partners/admins lock or unlock; sign-offs only as yourself at your level	`checkRules`
Adjustments proposed as yourself; approved by a reviewer+ who didn't propose them	`checkRules`
Practice rules need manager approval; archive only via the API	`checkRules`

API reference (Phase A)

Method	Path	Who	Notes
POST	`/api/v1/auth/signup`	Public	Creates firm, owner, starter client
POST	`/api/v1/auth/login`	Public	Session, or MFA challenge
GET	`/api/v1/auth/mfa/setup`	MFA challenge	Enrolment secret
POST	`/api/v1/auth/mfa/verify`	MFA challenge	`{ code }` → session
GET/POST	`/api/v1/invites`	Admin, partner, owner	List / create (returns link)
GET/POST	`/api/v1/invites/accept`	Invite token	Describe / accept
PUT	`/api/v1/users/<id>/clients`	Admin, partner, owner	Set assigned clients
GET/PUT	`/api/v1/firm/onboarding`	Admin, partner, owner (PUT)	Wizard state / save step
POST	`/api/v1/firm/onboarding/complete`	Admin, partner, owner	Finish set-up
GET/POST	`/api/v1/clients`	Member	Assigned clients only
POST	`/api/v1/clients/<id>/archive`	Manager+, owner	`{ archived, reason? }`
POST	`/api/v1/engagements`	Member	Not for archived/unassigned clients
POST	`/api/v1/engagements/<id>/imports`	Assigned	TB import, diff, mapping
GET	`/api/v1/engagements/<id>/mappings`	Assigned	
PATCH	`/api/v1/mappings/<id>`	Assigned	Correct → client rule
POST	`/api/v1/mappings/confirm`	Assigned	Confirm selected
POST	`/api/v1/rules/<id>/approve`	Manager+	Approve practice rule
GET	`/api/v1/ai/accuracy`	Member	Per output type
GET/PUT	`/api/v1/state`	Member	Workspace sync through the guard

Errors return `{ error, code }` with 400 (invalid input), 401 (not signed in), 403 (not allowed), 404 (not found or not yours), 409 (conflict, e.g. archived).

Testing

File	Covers
`src/__tests__/stage-a.test.ts`	Every Stage A "Done when": invite → MFA → session → assigned clients; audit immutability; client/engagement creation; late-posting detection; correction → rule; accuracy; second output type through the gate; code/name collision; client archive
`src/__tests__/onboarding.test.ts`	Sign-up → MFA challenge, wizard validation and completion, who may run set-up
`src/__tests__/validation.test.ts`	Email and password rules
`src/__tests__/auth.test.ts`	Passwords, sessions, firm isolation, role rules on sync
`src/__tests__/acceptance.test.ts`	Ground rules and earlier acceptance criteria

Route tests mock `@/lib/server/store`, so they never touch your workspace file or database.

Not done yet

Item	Step	Needs
Xero and QuickBooks connections (60-second pull, links back to the ledger)	3	Xero / Intuit developer apps and OAuth credentials
Email delivery of invites	1	An email service; links are copied by hand today
Logo upload, Companies House look-up in the wizard	2	File-storage decision; Companies House API key
Credit balances described from a debit view in change detection ("decreased" when Share Capital rises)	3	Wording change based on the account's normal balance
Turning MFA on	1	Set `MFA_REQUIRED=true` before launch